Cross-model consensus for solar filament instances

Technical report draft | Kaggle account: ranjeet258 | 28 October 2026

Draft status. This four-page document records the implemented experiment and measured results. Author identity and contact details are awaiting confirmation. The final report must be transferred to the organizers' specified Overleaf template before the final form is submitted.

Abstract

We combine three semantic U-Nets with YOLO11m instance proposals for 2048 x 2048 H-alpha observations. A learned selector uses geometry, soft foreground support and cross-model agreement to predict each mask's contribution to panoptic quality (PQ). We compare direct instance fusion with boundary-refined proposals and semantic-only selection. In the saved Kaggle execution, the chosen semantic-mask configuration achieved local held-out PQ 0.4525; the cross-validated settings estimate was 0.4511. These are local estimates, not public leaderboard scores. Direct addition of YOLO masks reduced validation performance and was rejected.

1. Data and objective

The competition supplies 707 training observations, often with multiple independent annotator sets, and 180 test observations. We retain each annotator set when computing target support and evaluation. A match requires IoU strictly greater than 0.5. PQ pools matched IoU over TP + 0.5 FP + 0.5 FN, penalizing false positives, missed instances and poor boundaries.

2. Model inputs

Family	Role
U-Net x3	ResNet34, ResNeXt50-32x4d and EfficientNet-B4; four folds each; soft shape/presence outputs.
YOLO11m-seg	Whole-instance proposals; network fold 0 is held out.
LightGBM	Four day-grouped meta-validation models; scores averaged on test candidates.

Only competition training annotations supervise the networks. The model-only notebook does not consume the historical public reference CSV.

3. Candidate generation and selection

We average semantic probability maps from the three U-Net families and extract connected components at foreground probability 0.5, retaining masks with area at least 60 pixels. YOLO proposals are retained from confidence 0.03. We generate a second YOLO proposal by preserving an eroded core and accepting semantic foreground only within a two-pixel boundary band.

The selector uses 19 features: proposal source and confidence, area and bounding-box geometry, normalized radius, mean/lower-quartile/maximum shape and presence probability, agreement with semantic and YOLO masks, matched YOLO confidence, boundary change and candidate count. Its target is the summed IoU of qualifying ground-truth matches divided by the number of annotator sets. The fit uses annotation-count weights.

We train LightGBM regressors with 250 boosting rounds, learning rate 0.035, 15 leaves, minimum leaf count 60 and L2 penalty 10. Four selectors are trained with observations from the same day kept together. Candidate sources and score cuts from 0.18 to 0.40 are compared using the exact PQ of the resulting pixel-disjoint masks.

4. Pixel-disjoint export

Candidates are ordered by predicted utility. A proposal losing more than 10% of its area to accepted instances is rejected; smaller overlaps are clipped. The resulting masks are written as compressed COCO RLE counts, one row per filament. We verify schema, unique IDs, nonempty masks, test-image membership and zero exact overlap using RLE unions.

5. Validation design and limits

YOLO never saw the 177 images in network fold 0 during its training. All selector learning and validation is restricted to that fold, avoiding the other images YOLO trained on. Meta-validation groups by observation day, although the original network folds group by physical image rather than day. The configuration estimate uses cross-fitted selector predictions but does not retrain the selectors inside a complete second nested loop. It can therefore remain optimistic. No test labels are used for local selection.

The local workstation and the pinned Kaggle environment differ in package versions, so they selected different score cuts. Each environment's result is preserved separately; the submitted model CSV comes from the completed Kaggle notebook.

EXPERIMENTAL EVIDENCE

6. Results and submission provenance

Candidate source	Best local PQ
semantic	0.4537 at cut 0.22
raw hybrid	0.4463 at cut 0.22
refined hybrid	0.4501 at cut 0.22
all	0.4489 at cut 0.22

The model-only saved Kaggle run selected semantic masks at cut 0.18. It exported 1237 non-overlapping masks. The workstation run selected cut 0.22 and exported 1142 masks. This difference is recorded rather than presenting the two executions as bitwise-identical.

Public-score receipts

Submission	Status / public PQ
boundary reference (ref 57745583)	PENDING
consensus reference (ref 57745523)	PENDING
model only (ref 57745515)	PENDING

Reference experiments are separate

The historical 0.55 submission relies on lamhuy8904's embedded prediction CSV; its generating weights are unavailable in that notebook. We do not attribute that score to this trained pipeline. A separately labelled consensus experiment adds 44 proposals supported by both model families to that reference. An additional boundary-band experiment changes only 34 masks, at most 10% of each mask's pixels.

Boundary refinement improved summed matched IoU by 31.07 over 367 strongly agreed held-out YOLO proposal pairs. This supports testing the operation on model proposals; it does not directly validate modifications to the stronger reference masks. Reference variants remain supplementary and do not satisfy independent reference-model reproduction.

7. Reproduction package

The source release contains the executable notebook, fusion implementation, all four original training scripts, pinned post-processing requirements, a proposed fresh-training environment, local and Kaggle validation reports, exact CSV hashes and Kaggle submission receipts. Training seeds are fixed, but the historic training scripts installed packages dynamically; their original dependency versions are not recoverable from source alone.

Cached reproduction

Run the saved Kaggle notebook with the competition and the outputs of ranjeet258/sf-seg-train-v1, sf-seg-train-v2, sf-seg-train-v3 and sf-yolo-v1 attached. It installs pinned numerical dependencies, constructs candidates, performs selector validation and writes submission.csv. This route requires those trained-output inputs to be accessible to the committee. Their public accessibility must be checked before the final application.

Fresh training

Run each supplied training script in a separate GPU session with competition data attached. Preserve weights, out-of-fold maps, test maps and the fold metadata. The U-Nets train four folds with multi-annotator soft targets, EMA and eight-view test-time augmentation; YOLO trains one sample per annotator set with network fold 0 excluded. Attach the generated outputs to the fusion notebook. Fresh training takes several GPU sessions and may not be bitwise-identical to the historic runs.

Outstanding final application items

Confirm author/team identity, affiliation and contact details; transfer this draft to the official four-page template; ensure the repository and necessary trained-output inputs are publicly accessible; complete the organizers' Google Form by November 15, 2026. A completed Kaggle notebook and an accepted prediction CSV do not by themselves complete the final judging application.

References

[1] Competition instructions: kaggle.com/competitions/filament-segmentation-2026/overview/announcements
[2] Official self-evaluator: kaggle.com/code/azimahmadzadeh/self-evaluation-notebook
[3] Kirillov et al. Panoptic Segmentation. CVPR 2019. doi:10.1109/CVPR.2019.00963
[4] Report template: overleaf.com/read/vjztpvrnbrdh
[5] Segmentation Models PyTorch: github.com/qubvel-org/segmentation_models.pytorch
[6] Ultralytics: github.com/ultralytics/ultralytics

AI assistance was used for implementation and documentation. The participant remains responsible for understanding and defending the method.